Full Python Django Web Development Course

Practical Guide — From Zero to Deployment

Using the Michael Aluminum & Glass Technology Project

12 Modules • Real-World Examples • Exercises • Cheat Sheets

Prepared by Getachew Zemicheal
github.com/getachewzemichael

July 2026

Practical Guide — Using Michael Aluminum & Glass Technology Project

From Zero to Deployment

> ***How to use this guide:*** *Every concept is explained with real examples from the Michael Aluminum project you already built. Open the project files as you read.*

TABLE OF CONTENTS

MODULE 1 — Python Fundamentals

1.1 Variables and Data Types

Python is dynamically typed — you don't declare types.

```
# Strings
company_name = "Michael Aluminum and Glass Technology"
city = 'Addis Ababa'

# Numbers
year = 2025
price = 150000.50

# Boolean
is_active = True
is_featured = False

# None (like null)
founder_photo = None
```

1.2 Lists, Tuples, Dictionaries, Sets

```
# List — ordered, changeable
categories = ['Handrail', 'ACP Cladding', 'Curtain Wall', 'Glass Partition']
categories.append('LTZ Windows')
categories[0]  # 'Handrail'

# Tuple — ordered, unchangeable
coordinates = (9.0107, 38.7612)  # Bole, Addis Ababa

# Dictionary — key-value pairs
project = {
    'title': 'Handrail Project',
    'location': 'Addis Ababa',
    'year': 2025,
    'is_active': True,
}
project['title']       # 'Handrail Project'
project.get('client')  # None (safe access)

# Set — unique values only
materials = {'aluminum', 'glass', 'steel', 'aluminum'}
# Result: {'aluminum', 'glass', 'steel'}
```

1.3 Conditionals

```
year = 2025

if year &gt;= 2024:
    print("Recent project")
elif year &gt;= 2020:
    print("Older project")
else:
    print("Archive project")

# Ternary
status = "active" if is_active else "inactive"
```

1.4 Loops

```
# For loop
categories = ['Handrail', 'ACP Cladding', 'Curtain Wall']
for category in categories:
    print(f"Category: {category}")

# For with index
for i, category in enumerate(categories):
    print(f"{i+1}. {category}")

# While loop
count = 0
while count &lt; 5:
    print(count)
    count += 1

# List comprehension (Pythonic way)
slugs = [cat.lower().replace(' ', '-') for cat in categories]
# ['handrail', 'acp-cladding', 'curtain-wall']
```

1.5 Functions

```
def create_slug(title):
    """Convert a title to a URL-friendly slug"""
    return title.lower().replace(' ', '-')

# With default parameter
def get_projects(limit=6, featured=True):
    # Returns projects from database
    pass

# With *args (variable positional arguments)
def add_categories(*category_names):
    for name in category_names:
        print(f"Adding: {name}")

# With **kwargs (variable keyword arguments)
def create_project(**data):
    title = data.get('title', 'Untitled')
    year = data.get('year', 2025)
    return f"{title} ({year})"

create_project(title='Handrail Project', year=2025, location='Addis Ababa')
```

1.6 Classes and OOP

```
class Project:
    """Represents a construction project"""

    # Class variable (shared by all instances)
    company = "Michael Aluminum"

    def __init__(self, title, category, year):
        # Instance variables
        self.title = title
        self.category = category
        self.year = year
        self.is_active = True

    def __str__(self):
        return f"{self.title} ({self.year})"

    def get_slug(self):
        return self.title.lower().replace(' ', '-')

    @classmethod
    def create_from_dict(cls, data):
        return cls(data['title'], data['category'], data['year'])

    @staticmethod
    def validate_year(year):
        return 2000 &lt;= year &lt;= 2030


# Inheritance
class FeaturedProject(Project):
    def __init__(self, title, category, year, priority=1):
        super().__init__(title, category, year)
        self.priority = priority
        self.is_featured = True


# Usage
p = Project('Handrail Project', 'Handrail', 2025)

print(p)           # Handrail Project (2025)
print(p.get_slug()) # handrail-project
```

1.7 Virtual Environments

```
# Create virtual environment
python -m venv venv

# Activate (Windows)
venv\Scripts\activate

# Activate (Mac/Linux)
source venv/bin/activate

# Install packages
pip install django

# Save dependencies
pip freeze &gt; requirements.txt

# Install from requirements
pip install -r requirements.txt

# Deactivate
deactivate
```

Project Exercise 1: Open `projects/models.py` in the Michael Aluminum project. Identify each Python concept used: classes, `__str__`, `__init__` equivalent, default values.

MODULE 2 — Django Basics

2.1 What is Django?

Django is a high-level Python web framework. It follows the **MVT pattern**:

Layer	Django Name	Responsibility
Model	`models.py`	Database structure and logic
View	`views.py`	Business logic, request handling
Template	`templates/`	HTML presentation

The URL router (`urls.py`) connects incoming requests to views.

2.2 Creating a Django Project

```
# Install Django
pip install django

# Create project
django-admin startproject MichaelAluminum

# Project structure created:
MichaelAluminum/
    manage.py              # CLI tool
    MichaelAluminum/
        __init__.py
        settings.py        # All configuration
        urls.py            # Root URL routing
        wsgi.py            # Web server interface
        asgi.py            # Async server interface
```

2.3 Creating Apps

Django projects are made of **apps** — each app handles one feature area.

```
# Create an app
python manage.py startapp projects

# App structure:
projects/
    __init__.py
    admin.py        # Admin panel config
    apps.py         # App configuration
    models.py       # Database models
    views.py        # Request handlers
    urls.py         # App URL patterns (you create this)
    tests.py        # Unit tests
    migrations/     # Database change history
```

In the Michael Aluminum project, we have 12 apps:

- `core` — home page, about, company info
- `services` — service listings
- `projects` — portfolio
- `gallery` — image gallery
- `blog` — blog posts
- `testimonials` — client reviews
- `careers` — job listings
- `contact` — contact form
- `quotations` — quote requests
- `accounts` — user management
- `dashboard` — admin dashboard
- `api` — REST API

2.4 settings.py — Key Settings

```
# MichaelAluminum/settings.py

# Security
SECRET_KEY = 'your-secret-key'   # Never share this!
DEBUG = True                     # False in production
ALLOWED_HOSTS = ['localhost']    # Which domains can access

# Installed apps — every app must be registered here
INSTALLED_APPS = [
    'django.contrib.admin',        # Admin panel
    'django.contrib.auth',         # Authentication
    'django.contrib.contenttypes', # Content types framework
    'django.contrib.sessions',     # Session management
    'django.contrib.messages',     # Flash messages
    'django.contrib.staticfiles',  # Static file serving
    # Third party
    'rest_framework',
    'corsheaders',
    # Our apps
    'core',
    'services',
    'projects',
    # ...
]

# Database
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',  # Development
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}

# Static files
STATIC_URL = '/static/'
STATIC_ROOT = BASE_DIR / 'staticfiles'
STATICFILES_DIRS = [BASE_DIR / 'static']

# Media files
MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'media'
```

2.5 manage.py Commands

```
# Run development server
python manage.py runserver

# Run on specific port
python manage.py runserver 0.0.0.0:8080

# Create database migrations
python manage.py makemigrations

# Apply migrations to database
python manage.py migrate

# Create admin user
python manage.py createsuperuser

# Open Django shell (Python + Django)
python manage.py shell

# Collect static files for production
python manage.py collectstatic

# Check for issues
python manage.py check

# Run custom management command
python manage.py seed_services
```

Project Exercise 2: Run `python manage.py shell` and type:

```
from projects.models import Project
Project.objects.all()
Project.objects.count()
Project.objects.first()
```

MODULE 3 — Models & Database

3.1 What is an ORM?

ORM (Object Relational Mapper) lets you work with the database using Python classes instead of writing SQL.

```
# Without ORM (raw SQL)
cursor.execute("SELECT * FROM projects WHERE is_active = 1 AND year = 2025")

# With Django ORM
Project.objects.filter(is_active=True, year=2025)
```

3.2 Creating Models

Open `projects/models.py` in the Michael Aluminum project:

```
from django.db import models
from django.utils.text import slugify

class ProjectCategory(models.Model):
    name = models.CharField(max_length=100)     # Short text
    slug = models.SlugField(unique=True)         # URL-friendly text, must be unique
    description = models.TextField(blank=True)   # Long text, optional
    icon = models.CharField(max_length=100, blank=True)
    order = models.IntegerField(default=0)       # Number with default

    class Meta:
        ordering = ['order']                     # Default sort order
        verbose_name = "Project Category"
        verbose_name_plural = "Project Categories"

    def __str__(self):
        return self.name                         # How it shows in admin


class Project(models.Model):
    # ForeignKey = many projects can belong to one category
    category = models.ForeignKey(
        ProjectCategory,
        on_delete=models.SET_NULL,  # If category deleted, set to NULL
        null=True,
        related_name='projects'     # Access via category.projects.all()
    )
    title = models.CharField(max_length=200)
    slug = models.SlugField(unique=True)
    description = models.TextField()

    # Image fields
    featured_image = models.ImageField(
        upload_to="projects/featured/",  # Where to save
        blank=True, null=True
    )

    # Static image path (for Render deployment)
```

```
    static_featured = models.CharField(max_length=255, blank=True)

    # Project details
    location = models.CharField(max_length=300)
    client = models.CharField(max_length=200)
    year = models.IntegerField()

    # Boolean flags
    is_featured = models.BooleanField(default=False)
    is_active = models.BooleanField(default=True)

    # Auto timestamps
    created_at = models.DateTimeField(auto_now_add=True)  # Set once on creation
    updated_at = models.DateTimeField(auto_now=True)       # Updated every save

    def save(self, *args, **kwargs):
        # Auto-generate slug from title
        if not self.slug:
            self.slug = slugify(self.title)
        super().save(*args, **kwargs)
```

3.3 Field Types Reference

Field	Use	Example
`CharField(max_length=n)`	Short text	names, titles
`TextField()`	Long text	descriptions, content
`IntegerField()`	Whole numbers	year, order, rating
`FloatField()`	Decimal numbers	price, rating
`BooleanField()`	True/False	is_active, is_featured
`DateField()`	Date only	birth_date
`DateTimeField()`	Date + time	created_at
`EmailField()`	Validated email	contact email
`URLField()`	Validated URL	website, social
`SlugField()`	URL-safe text	handrail-project
`ImageField()`	Image upload	photos
`FileField()`	Any file upload	documents
`ForeignKey()`	Many-to-one relation	project → category
`ManyToManyField()`	Many-to-many relation	tags
`OneToOneField()`	One-to-one relation	user → profile

3.4 Migrations

Every time you change a model, you must create and apply a migration:

```
# Step 1: Create migration file
python manage.py makemigrations

# Step 2: Apply to database
python manage.py migrate

# See migration history
python manage.py showmigrations

# See the SQL that would run
python manage.py sqlmigrate projects 0001
```

Migration file example (`projects/migrations/0001_initial.py`):

```
from django.db import migrations, models

class Migration(migrations.Migration):
    initial = True
    dependencies = []
    operations = [
        migrations.CreateModel(
            name='ProjectCategory',
            fields=[
                ('id', models.BigAutoField(primary_key=True)),
                ('name', models.CharField(max_length=100)),
                ('slug', models.SlugField(unique=True)),
            ],
        ),
    ]
```

3.5 QuerySet — Querying the Database

```
from projects.models import Project, ProjectCategory

# Get all projects
Project.objects.all()

# Filter
Project.objects.filter(is_active=True)
Project.objects.filter(year=2025, is_featured=True)

# Exclude
Project.objects.exclude(is_active=False)

# Get single object (raises error if not found)
Project.objects.get(slug='handrail-project-1')

# Get or return 404
from django.shortcuts import get_object_or_404
project = get_object_or_404(Project, slug='handrail-project-1')

# Order by
Project.objects.all().order_by('year')        # ascending
Project.objects.all().order_by('-year')       # descending

# Limit results
Project.objects.all()[:6]                     # first 6

# Count
Project.objects.filter(is_active=True).count()

# Check if exists
Project.objects.filter(slug='handrail').exists()

# Complex queries with Q
from django.db.models import Q
Project.objects.filter(
    Q(title__icontains='handrail') | Q(location__icontains='bole')
)
```

```
# Related data — select_related (JOIN, one query)
Project.objects.select_related('category').filter(is_active=True)

# Create
Project.objects.create(
    title='New Project',
    slug='new-project',
    year=2025,
    is_active=True
)

# Update
Project.objects.filter(year=2024).update(is_active=False)

# Delete
Project.objects.filter(is_active=False).delete()

# Get or create
project, created = Project.objects.get_or_create(
    slug='handrail-project',
    defaults={'title': 'Handrail Project', 'year': 2025}
)

# Update or create
project, created = Project.objects.update_or_create(
    slug='handrail-project',
    defaults={'title': 'Updated Title', 'year': 2025}
)
```

3.6 Field Lookups

```
# Exact match (default)
Project.objects.filter(year=2025)
Project.objects.filter(year__exact=2025)

# Contains (case-sensitive)
Project.objects.filter(title__contains='Handrail')

# icontains (case-insensitive)
Project.objects.filter(title__icontains='handrail')

# Starts with
Project.objects.filter(title__startswith='Handrail')

# Greater than / Less than
Project.objects.filter(year__gt=2022)   # &gt;
Project.objects.filter(year__gte=2022)  # &gt;=
Project.objects.filter(year__lt=2025)   # &lt;
Project.objects.filter(year__lte=2025)  # &lt;=

# In a list
Project.objects.filter(year__in=[2023, 2024, 2025])

# Is null
Project.objects.filter(featured_image__isnull=True)

# Related field lookup (double underscore)
Project.objects.filter(category__name='Handrail')
Project.objects.filter(category__slug='handrail')
```

Project Exercise 3: Open Django shell and practice:

```
from projects.models import Project
# Try each of the above queries
# Add a new field to the Project model, make migrations, see it in admin
```

MODULE 4 — Views & URLs

4.1 How a Request Works in Django

```
Browser requests /projects/handrail-project-1/
        ↓
MichaelAluminum/urls.py  → matches "projects/"
        ↓
projects/urls.py         → matches "&lt;slug:slug&gt;/"
        ↓
projects/views.py        → project_detail() function runs
        ↓
Queries database, builds context
        ↓
templates/projects/project_detail.html renders
        ↓
HTML response sent back to browser
```

4.2 Root URLs — urls.py

```
# MichaelAluminum/urls.py
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path("admin/", admin.site.urls),
    # include() delegates to each app's urls.py
    path("", include("core.urls", namespace="core")),
    path("services/", include("services.urls", namespace="services")),
    path("projects/", include("projects.urls", namespace="projects")),
    path("blog/", include("blog.urls", namespace="blog")),
    path("contact/", include("contact.urls", namespace="contact")),
]
```

4.3 App URLs

```
# projects/urls.py
from django.urls import path
from . import views

app_name = 'projects'  # Namespace for {% url 'projects:list' %}

urlpatterns = [
    path('', views.projects_list, name='list'),
    path('&lt;slug:slug&gt;/', views.project_detail, name='detail'),
]
```

4.4 URL Patterns

```
# Exact match
path('about/', views.about, name='about')
# Matches: /about/

# Integer parameter
path('projects/&lt;int:pk&gt;/', views.project_detail, name='detail')
# Matches: /projects/1/  /projects/42/

# Slug parameter
path('projects/&lt;slug:slug&gt;/', views.project_detail, name='detail')
# Matches: /projects/handrail-project/

# String parameter
path('users/&lt;str:username&gt;/', views.profile, name='profile')
# Matches: /users/michael/

# UUID parameter
path('orders/&lt;uuid:order_id&gt;/', views.order, name='order')
```

4.5 Function-Based Views (FBV)

```
# projects/views.py
from django.shortcuts import render, get_object_or_404
from django.db.models import Q
from .models import Project, ProjectCategory


def projects_list(request):
    """Handle GET /projects/"""

    # Get query parameters from URL
    # e.g. /projects/?category=handrail&amp;q=bole
    category = request.GET.get('category')
    search = request.GET.get('q')

    # Build queryset
    projects = Project.objects.select_related('category').filter(
        is_active=True
    ).exclude(static_featured='')

    # Apply filters if provided
    if category:
        projects = projects.filter(category__slug=category)

    if search:
        projects = projects.filter(
            Q(title__icontains=search) |
            Q(description__icontains=search)
        )

    categories = ProjectCategory.objects.all()

    # Context is a dictionary passed to the template
    context = {
        'projects': projects,
        'categories': categories,
        'selected_category': category,
        'search_query': search,
    }
```

```
    # Render template with context
    return render(request, 'projects/projects_list.html', context)


def project_detail(request, slug):
    """Handle GET /projects/&lt;slug&gt;/"""

    # Get object or return 404 error page
    project = get_object_or_404(
        Project.objects.select_related('category'),
        slug=slug,
        is_active=True
    )

    related = Project.objects.filter(
        category=project.category,
        is_active=True
    ).exclude(id=project.id)[:3]

    context = {
        'project': project,
        'related_projects': related,
    }
    return render(request, 'projects/project_detail.html', context)
```

4.6 HttpResponse Types

```
from django.shortcuts import render, redirect
from django.http import HttpResponse, JsonResponse, Http404

# Render HTML template
return render(request, 'template.html', context)

# Redirect to another URL
return redirect('/projects/')
return redirect('projects:list')   # Using URL namespace

# Return plain text
return HttpResponse("Hello World")

# Return JSON (for APIs)
return JsonResponse({'status': 'ok', 'count': 6})

# Return 404
raise Http404("Project not found")

# Return 404 (easier way)
from django.shortcuts import get_object_or_404
obj = get_object_or_404(Project, slug=slug)
```

4.7 Request Object

```
def my_view(request):
    # HTTP method
    request.method         # 'GET', 'POST', 'PUT', 'DELETE'

    # GET parameters (?key=value in URL)
    request.GET.get('category')
    request.GET.get('page', 1)  # Default value

    # POST data (form submissions)
    request.POST.get('name')
    request.POST.get('email')

    # Uploaded files
    request.FILES.get('image')

    # Current user
    request.user
    request.user.is_authenticated
    request.user.username

    # Session data
    request.session['cart'] = []

    # Full URL info
    request.path            # '/projects/'
    request.get_full_path() # '/projects/?category=handrail'
    request.META['HTTP_HOST']  # 'michael-aluminum.onrender.com'
```

4.8 URL Namespacing — Using {% url %}

```
# In templates
{% url 'projects:list' %}
# Output: /projects/

{% url 'projects:detail' project.slug %}
# Output: /projects/handrail-project-1/

{% url 'blog:detail' post.slug %}
{% url 'core:home' %}
{% url 'contact:contact' %}

# In views (Python)
from django.urls import reverse
url = reverse('projects:detail', kwargs={'slug': 'handrail-project'})
# Output: '/projects/handrail-project/'

# Redirect using name
from django.shortcuts import redirect
return redirect('projects:list')
```

Project Exercise 4: Open `core/urls.py`, `projects/urls.py`. Trace the full path of the URL `/projects/handrail-project-1/` from `MichaelAluminum/urls.py` → `projects/urls.py` → `project_detail()` view. Add a new URL `path('featured/', views.featured_projects, name='featured')` and create the view.

MODULE 5 — Templates

5.1 Template Syntax

Django templates use `{{ }}` for variables and `{% %}` for logic.

```
&lt;!-- Variable output --&gt;
&lt;h1&gt;{{ project.title }}&lt;/h1&gt;
&lt;p&gt;{{ project.description }}&lt;/p&gt;
&lt;small&gt;{{ project.year }}&lt;/small&gt;

&lt;!-- Attribute access --&gt;
{{ project.category.name }}
{{ request.user.username }}
```

5.2 Template Filters

```
&lt;!-- Lowercase --&gt;
{{ project.title|lower }}

&lt;!-- Uppercase --&gt;
{{ project.title|upper }}

&lt;!-- Truncate to 50 chars --&gt;
{{ project.description|truncatechars:50 }}

&lt;!-- Truncate words --&gt;
{{ post.content|truncatewords:20 }}

&lt;!-- Default if empty --&gt;
{{ project.client|default:"Unknown Client" }}

&lt;!-- Date formatting --&gt;
{{ project.created_at|date:"M d, Y" }}
{{ post.published_at|date:"F j, Y" }}

&lt;!-- Length --&gt;
{{ categories|length }}

&lt;!-- Slugify --&gt;
{{ project.title|slugify }}

&lt;!-- Safe (render HTML without escaping) --&gt;
{{ post.content|safe }}

&lt;!-- Line breaks to HTML --&gt;
{{ project.description|linebreaks }}

&lt;!-- Join list --&gt;
{{ materials|join:", " }}

&lt;!-- Add --&gt;
{{ forloop.counter|add:100 }}
```

5.3 Template Tags

```
&lt;!-- For loop --&gt;
{% for project in projects %}
    &lt;div&gt;{{ project.title }}&lt;/div&gt;
{% empty %}
    &lt;p&gt;No projects found.&lt;/p&gt;
{% endfor %}

&lt;!-- For loop with counter --&gt;
{% for project in projects %}
    {{ forloop.counter }}     {# 1, 2, 3... #}
    {{ forloop.counter0 }}    {# 0, 1, 2... #}
    {{ forloop.first }}       {# True on first iteration #}
    {{ forloop.last }}        {# True on last iteration #}
{% endfor %}

&lt;!-- If/elif/else --&gt;
{% if project.is_featured %}
    &lt;span class="badge"&gt;Featured&lt;/span&gt;
{% elif project.is_active %}
    &lt;span class="badge"&gt;Active&lt;/span&gt;
{% else %}
    &lt;span class="badge"&gt;Inactive&lt;/span&gt;
{% endif %}

&lt;!-- If with comparison --&gt;
{% if projects|length &gt; 3 %}
    &lt;p&gt;Many projects&lt;/p&gt;
{% endif %}

&lt;!-- If with and/or/not --&gt;
{% if user.is_authenticated and user.is_staff %}
    &lt;a href="/admin/"&gt;Admin&lt;/a&gt;
{% endif %}

&lt;!-- URL tag --&gt;
&lt;a href="{% url 'projects:detail' project.slug %}"&gt;View&lt;/a&gt;

&lt;!-- Static tag --&gt;

{% load static %}
&lt;img src="{% static 'images/logo.jpg' %}" alt="Logo"&gt;

&lt;!-- Include another template --&gt;
{% include 'partials/navbar.html' %}
{% include 'partials/footer.html' with company=company_info %}

&lt;!-- Block tag (for inheritance) --&gt;
{% block title %}Default Title{% endblock %}
{% block content %}{% endblock %}

&lt;!-- CSRF token (required in all forms) --&gt;
&lt;form method="post"&gt;
    {% csrf_token %}
    ...
&lt;/form&gt;
```

5.4 Template Inheritance

The most powerful feature of Django templates.

```
&lt;!-- templates/base.html — the parent template --&gt;
&lt;!DOCTYPE html&gt;
&lt;html&gt;
&lt;head&gt;
    &lt;title&gt;{% block title %}Michael Aluminum{% endblock %}&lt;/title&gt;
    {% load static %}
    &lt;link rel="stylesheet" href="{% static 'css/style.css' %}"&gt;
    {% block extra_css %}{% endblock %}
&lt;/head&gt;
&lt;body&gt;
    {% include 'partials/navbar.html' %}

    &lt;main id="main-content"&gt;
        {% block content %}{% endblock %}
    &lt;/main&gt;

    {% include 'partials/footer.html' %}

    &lt;script src="{% static 'js/main.js' %}"&gt;&lt;/script&gt;
    {% block extra_js %}{% endblock %}
&lt;/body&gt;
&lt;/html&gt;
```

```
&lt;!-- templates/projects/projects_list.html — child template --&gt;
{% extends 'base.html' %}
{% load static %}

{% block title %}Our Projects - Michael Aluminum{% endblock %}

{% block extra_css %}
&lt;link rel="stylesheet" href="{% static 'css/projects.css' %}"&gt;
{% endblock %}

{% block content %}
&lt;section class="page-header"&gt;
    &lt;h1&gt;Our Projects&lt;/h1&gt;
&lt;/section&gt;

&lt;div class="row"&gt;
    {% for project in projects %}
    &lt;div class="col-md-4"&gt;
        {% if project.static_featured %}
        &lt;img loading="lazy" src="{% static project.static_featured %}" alt="{{ project.title }}"&
        {% endif %}
        &lt;h3&gt;{{ project.title }}&lt;/h3&gt;
        &lt;a href="{% url 'projects:detail' project.slug %}"&gt;View Details&lt;/a&gt;
    &lt;/div&gt;
    {% endfor %}
&lt;/div&gt;
{% endblock %}
```

5.5 Context Processors

Context processors add variables to every template automatically.

```
# settings.py
TEMPLATES = [{
    'OPTIONS': {
        'context_processors': [
            'django.template.context_processors.request',    # request
            'django.contrib.auth.context_processors.auth',   # user
            'django.contrib.messages.context_processors.messages',  # messages
            'django.template.context_processors.media',      # MEDIA_URL
        ],
    },
}]
```

```
&lt;!-- Available in ALL templates without passing in context --&gt;
{{ request.user }}
{{ request.path }}
{{ MEDIA_URL }}
{% if messages %}
    {% for message in messages %}
        &lt;div class="alert alert-{{ message.tags }}"&gt;{{ message }}&lt;/div&gt;
    {% endfor %}
{% endif %}
```

5.6 Custom Template Tag (Real Example from Project)

```
# projects/templatetags/project_tags.py
from django import template
register = template.Library()

@register.filter
def get_item(dictionary, key):
    return dictionary.get(key)

@register.simple_tag
def get_featured_projects(count=3):
    from projects.models import Project
    return Project.objects.filter(is_active=True, is_featured=True)[:count]
```

```
&lt;!-- Usage in template --&gt;
{% load project_tags %}
{% get_featured_projects 6 as featured %}
{% for project in featured %}
    {{ project.title }}
{% endfor %}
```

Project Exercise 5: Open `templates/core/home.html`. Find every `{% %}` tag and `{{ }}` variable. Modify the "Our Services" section heading to include the total count of services.

MODULE 6 — Forms & User Input

6.1 HTML Form with Django

```
&lt;!-- templates/contact/contact.html --&gt;
&lt;form method="post" action="{% url 'contact:contact' %}"&gt;
    {% csrf_token %}   &lt;!-- Required! Prevents CSRF attacks --&gt;

    &lt;div class="mb-3"&gt;
        &lt;label for="name"&gt;Full Name *&lt;/label&gt;
        &lt;input type="text" id="name" name="name" required&gt;
    &lt;/div&gt;

    &lt;div class="mb-3"&gt;
        &lt;label for="email"&gt;Email *&lt;/label&gt;
        &lt;input type="email" id="email" name="email" required&gt;
    &lt;/div&gt;

    &lt;div class="mb-3"&gt;
        &lt;label for="message"&gt;Message *&lt;/label&gt;
        &lt;textarea id="message" name="message" rows="5" required&gt;&lt;/textarea&gt;
    &lt;/div&gt;

    &lt;button type="submit"&gt;Send Message&lt;/button&gt;
&lt;/form&gt;
```

6.2 Handling the Form in a View

```
# contact/views.py
from django.shortcuts import render, redirect
from django.contrib import messages

def contact(request):
    if request.method == 'POST':
        # Get form data
        name = request.POST.get('name', '').strip()
        email = request.POST.get('email', '').strip()
        message = request.POST.get('message', '').strip()

        # Validate
        errors = {}
        if not name:
            errors['name'] = 'Name is required.'
        if not email:
            errors['email'] = 'Email is required.'
        if not message:
            errors['message'] = 'Message is required.'

        if not errors:
            # Save to database
            ContactMessage.objects.create(
                name=name,
                email=email,
                message=message
            )
            # Success flash message
            messages.success(request, 'Message sent successfully!')
            return redirect('contact:contact')
        else:
            # Re-render with errors
            return render(request, 'contact/contact.html', {
                'errors': errors,
                'form_data': request.POST,
            })

    # GET request — show empty form

    return render(request, 'contact/contact.html')
```

6.3 Django Form Class

```
# contact/forms.py
from django import forms

class ContactForm(forms.Form):
    name = forms.CharField(
        max_length=200,
        widget=forms.TextInput(attrs={'class': 'form-control', 'placeholder': 'Your name'})
    )
    email = forms.EmailField(
        widget=forms.EmailInput(attrs={'class': 'form-control'})
    )
    phone = forms.CharField(
        max_length=20,
        required=False,  # Optional field
        widget=forms.TextInput(attrs={'class': 'form-control'})
    )
    message = forms.CharField(
        widget=forms.Textarea(attrs={'class': 'form-control', 'rows': 5})
    )

    # Custom validation
    def clean_email(self):
        email = self.cleaned_data['email']
        if 'spam' in email:
            raise forms.ValidationError("Invalid email address.")
        return email

    def clean(self):
        cleaned_data = super().clean()
        name = cleaned_data.get('name')
        message = cleaned_data.get('message')
        if name and message and name in message:
            raise forms.ValidationError("Please provide a real message.")
        return cleaned_data
```

```
# contact/views.py using Form class
from .forms import ContactForm

def contact(request):
    if request.method == 'POST':
        form = ContactForm(request.POST)
        if form.is_valid():
            # Access cleaned (validated) data
            name = form.cleaned_data['name']
            email = form.cleaned_data['email']
            message = form.cleaned_data['message']
            # Process...
            messages.success(request, 'Message sent!')
            return redirect('contact:contact')
    else:
        form = ContactForm()  # Empty form for GET

    return render(request, 'contact/contact.html', {'form': form})
```

6.4 ModelForm — Auto-Generated Forms

```
# quotations/forms.py
from django import forms
from .models import Quotation

class QuotationForm(forms.ModelForm):
    class Meta:
        model = Quotation
        fields = ['client_name', 'email', 'phone', 'service_type', 'description']
        widgets = {
            'client_name': forms.TextInput(attrs={'class': 'form-control'}),
            'email': forms.EmailInput(attrs={'class': 'form-control'}),
            'description': forms.Textarea(attrs={'class': 'form-control', 'rows': 5}),
        }
        labels = {
            'client_name': 'Your Name',
            'service_type': 'Service Required',
        }
```

6.5 File Upload Forms

```
# forms.py
class ProjectImageForm(forms.Form):
    image = forms.ImageField()
    caption = forms.CharField(max_length=200, required=False)

# views.py
def upload_image(request):
    if request.method == 'POST':
        form = ProjectImageForm(request.POST, request.FILES)  # Include FILES!
        if form.is_valid():
            image = form.cleaned_data['image']
            # Save image
            instance = ProjectImage(image=image)
            instance.save()
    else:
        form = ProjectImageForm()
    return render(request, 'upload.html', {'form': form})
```

```
&lt;!-- Must include enctype for file uploads! --&gt;
&lt;form method="post" enctype="multipart/form-data"&gt;
    {% csrf_token %}
    &lt;input type="file" name="image" accept="image/*"&gt;
    &lt;button type="submit"&gt;Upload&lt;/button&gt;
&lt;/form&gt;
```

6.6 Flash Messages

```
# views.py
from django.contrib import messages

messages.success(request, 'Your message was sent successfully!')
messages.error(request, 'Something went wrong. Please try again.')
messages.warning(request, 'Please fill in all required fields.')
messages.info(request, 'Your quote request has been received.')
```

```
&lt;!-- base.html — show messages on every page --&gt;
{% if messages %}
    {% for message in messages %}
    &lt;div class="alert alert-{{ message.tags }}"&gt;
        {{ message }}
    &lt;/div&gt;
    {% endfor %}
{% endif %}
```

Project Exercise 6: Open `contact/views.py` and `templates/contact/contact.html`. Trace a form submission. Add a new field "phone number" to the contact form, validate that it starts with "+251".

MODULE 7 — Static Files & Media

7.1 Static vs Media Files

Type	What	Example	Setting
Static	Developer-created files	CSS, JS, logo.jpg	`STATIC_URL`
Media	User-uploaded files	project photos	`MEDIA_URL`

7.2 Static Files Configuration

```
# settings.py
STATIC_URL = '/static/'

# Where Django looks for static files during development
STATICFILES_DIRS = [BASE_DIR / 'static']

# Where collectstatic copies everything for production
STATIC_ROOT = BASE_DIR / 'staticfiles'

# WhiteNoise serves static files in production
STATICFILES_STORAGE = 'whitenoise.storage.CompressedManifestStaticFilesStorage'
```

```
static/
    css/
        style.css
        theme.css
        about.css
        footer.css
    js/
        main.js
    images/
        logo.jpg
        logo-nobg.png
        about-company.jpg
        Category 1 Handrail/
            photo_xxx_y.jpg
```

7.3 Using Static Files in Templates

```
{% load static %}

&lt;!-- CSS --&gt;
&lt;link rel="stylesheet" href="{% static 'css/style.css' %}"&gt;

&lt;!-- JavaScript --&gt;
&lt;script src="{% static 'js/main.js' %}"&gt;&lt;/script&gt;

&lt;!-- Images --&gt;
&lt;img src="{% static 'images/logo.jpg' %}" alt="Logo"&gt;

&lt;!-- Dynamic static path (from database) --&gt;
&lt;img src="{% static service.static_image %}" alt="{{ service.title }}"&gt;
```

7.4 Media Files Configuration

```
# settings.py
MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'media'
```

```
# urls.py — serve media in development only
from django.conf import settings
from django.conf.urls.static import static

if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

```
media/
    projects/
        featured/
            photo_xxx.jpg
        before/
            photo_xxx_before.jpg
        after/
            photo_xxx_after.jpg
    services/
        service_handrail.jpg
```

7.5 Accessing Media in Templates

```
&lt;!-- ImageField url property --&gt;
{% if project.featured_image %}
    &lt;img src="{{ project.featured_image.url }}" alt="{{ project.title }}"&gt;
{% endif %}

&lt;!-- Or use static_image field (Render-friendly) --&gt;
{% if project.static_featured %}
    &lt;img src="{% static project.static_featured %}" alt="{{ project.title }}"&gt;
{% endif %}
```

7.6 collectstatic for Production

```
# Copies all static files to STATIC_ROOT
python manage.py collectstatic --noinput

# Files are collected from:
# 1. Each app's static/ folder
# 2. STATICFILES_DIRS
# Into STATIC_ROOT (staticfiles/)
```

7.7 WhiteNoise — Static Files in Production

WhiteNoise serves static files directly from Django (no Nginx needed for static).

```
# settings.py
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'whitenoise.middleware.WhiteNoiseMiddleware',  # Must be second!
    ...
]

STATICFILES_STORAGE = 'whitenoise.storage.CompressedManifestStaticFilesStorage'
```

Important for Render: *Media files (user uploads) are NOT persistent on Render's free plan. They get wiped on every redeploy. That is why in the Michael Aluminum project we use* `static_image` *and* `static_featured` *fields to point to static files instead.*

Project Exercise 7: Add a new CSS file `static/css/projects-custom.css` with a custom style for project cards. Include it only on the projects list page using `{% block extra_css %}`.

MODULE 8 — Authentication & Users

8.1 Django's Built-in Auth System

Django ships with a complete authentication system out of the box.

```
# Already in INSTALLED_APPS:
'django.contrib.auth',         # Authentication framework
'django.contrib.contenttypes', # Required by auth
```

The default `User` model has:

- `username` — unique login name
- `email`
- `password` — hashed automatically
- `first_name`, `last_name`
- `is_active` — can login?
- `is_staff` — can access /admin/?
- `is_superuser` — all permissions

8.2 Login / Logout

```
# views.py
from django.contrib.auth import authenticate, login, logout
from django.shortcuts import render, redirect

def login_view(request):
    if request.method == 'POST':
        username = request.POST.get('username')
        password = request.POST.get('password')

        # Verify credentials
        user = authenticate(request, username=username, password=password)

        if user is not None:
            login(request, user)          # Create session
            return redirect('core:home')
        else:
            return render(request, 'accounts/login.html', {
                'error': 'Invalid username or password.'
            })

    return render(request, 'accounts/login.html')


def logout_view(request):
    logout(request)          # Destroy session
    return redirect('core:home')
```

8.3 Protecting Views

```
# Method 1: @login_required decorator
from django.contrib.auth.decorators import login_required

@login_required(login_url='/accounts/login/')
def my_profile(request):
    return render(request, 'profile.html')


# Method 2: @staff_member_required (admin/staff only)
from django.contrib.admin.views.decorators import staff_member_required

@staff_member_required
def dashboard(request):
    return render(request, 'dashboard/dashboard.html')


# Method 3: Manual check inside view
def secret_page(request):
    if not request.user.is_authenticated:
        return redirect('accounts:login')
    if not request.user.is_staff:
        return redirect('core:home')
    # ... rest of view
```

8.4 User in Templates

```
&lt;!-- Check if logged in --&gt;
{% if user.is_authenticated %}
    &lt;p&gt;Welcome, {{ user.username }}!&lt;/p&gt;
    &lt;a href="{% url 'accounts:logout' %}"&gt;Logout&lt;/a&gt;
{% else %}
    &lt;a href="{% url 'accounts:login' %}"&gt;Login&lt;/a&gt;
{% endif %}

&lt;!-- Check if staff --&gt;
{% if user.is_staff %}
    &lt;a href="/admin/"&gt;Admin Panel&lt;/a&gt;
{% endif %}
```

8.5 Custom User Model (Best Practice)

Always create a custom user model at the start of a project.

```
# accounts/models.py
from django.contrib.auth.models import AbstractUser
from django.db import models

class CustomUser(AbstractUser):
    # Add extra fields
    phone = models.CharField(max_length=20, blank=True)
    profile_photo = models.ImageField(upload_to='profiles/', blank=True, null=True)
    bio = models.TextField(blank=True)
    is_verified = models.BooleanField(default=False)

    def __str__(self):
        return self.username
```

```
# settings.py
AUTH_USER_MODEL = 'accounts.CustomUser'   # Tell Django to use our model
```

8.6 Creating Users Programmatically

```
from django.contrib.auth import get_user_model
User = get_user_model()

# Create regular user
user = User.objects.create_user(
    username='michael',
    email='michael@example.com',
    password='securepassword123'
)

# Create superuser
User.objects.create_superuser(
    username='admin',
    email='admin@example.com',
    password='adminpassword'
)

# Change password
user.set_password('newpassword')
user.save()

# Check password
user.check_password('newpassword')  # True
```

In the Michael Aluminum project: The `docker_start.sh` creates the admin superuser automatically on every Render deploy:

```
python manage.py shell -c "
from django.contrib.auth import get_user_model
User = get_user_model()
if not User.objects.filter(username='admin').exists():
    User.objects.create_superuser('admin', 'michaeltadessemiki@gmail.com', 'admin123')
"
```

Project Exercise 8: Create a new user via the Django shell. Login to `/admin/` with it. Restrict the `dashboard` view so only staff members can access it.

MODULE 9 — Django Admin

9.1 Registering Models

```
# projects/admin.py
from django.contrib import admin
from .models import Project, ProjectCategory, ProjectImage


# Simple registration
admin.site.register(Project)


# Full customization with decorator
@admin.register(Project)
class ProjectAdmin(admin.ModelAdmin):

    # Columns shown in list view
    list_display = ('title', 'category', 'year', 'is_active', 'is_featured')

    # Clickable link column
    list_display_links = ('title',)

    # Editable directly in list view
    list_editable = ('is_active', 'is_featured')

    # Sidebar filters
    list_filter = ('is_active', 'is_featured', 'category', 'year')

    # Search box
    search_fields = ('title', 'description', 'location', 'client')

    # Auto-fill slug from title
    prepopulated_fields = {'slug': ('title',)}

    # Date hierarchy navigation
    date_hierarchy = 'created_at'

    # Default sort
    ordering = ['-created_at']

    # How many items per page
```

```
    list_per_page = 20

    # Organize edit form into sections
    fieldsets = (
        ('Basic Information', {
            'fields': ('title', 'slug', 'category', 'description')
        }),
        ('Images', {
            'fields': ('featured_image', 'static_featured', 'before_image',
                       'static_before', 'after_image', 'static_after'),
            'classes': ('collapse',),  # Collapsible section
        }),
        ('Project Details', {
            'fields': ('location', 'client', 'year', 'duration', 'materials_used'),
        }),
        ('Client Feedback', {
            'fields': ('client_feedback', 'client_rating'),
            'classes': ('collapse',),
        }),
        ('Settings', {
            'fields': ('is_active', 'is_featured', 'order'),
        }),
    )

    # Read-only fields
    readonly_fields = ('created_at', 'updated_at')
```

9.2 Inline Admin (Edit Related Models Together)

```
# projects/admin.py
class ProjectImageInline(admin.TabularInline):
    model = ProjectImage
    extra = 2              # Show 2 empty forms
    max_num = 20

class ProjectVideoInline(admin.StackedInline):
    model = ProjectVideo
    extra = 1

@admin.register(Project)
class ProjectAdmin(admin.ModelAdmin):
    inlines = [ProjectImageInline, ProjectVideoInline]
    # ... rest of config
```

9.3 Custom Admin Actions

```
@admin.register(Project)
class ProjectAdmin(admin.ModelAdmin):
    actions = ['make_featured', 'make_inactive']

    @admin.action(description='Mark selected projects as featured')
    def make_featured(self, request, queryset):
        queryset.update(is_featured=True)
        self.message_user(request, f'{queryset.count()} projects marked as featured.')

    @admin.action(description='Deactivate selected projects')
    def make_inactive(self, request, queryset):
        queryset.update(is_active=False)
```

9.4 Custom Admin Display

```
@admin.register(Project)
class ProjectAdmin(admin.ModelAdmin):
    list_display = ('title', 'category', 'year', 'image_preview', 'is_active')

    def image_preview(self, obj):
        from django.utils.html import format_html
        if obj.static_featured:
            return format_html(
                '&lt;img src="/static/{}" style="height:50px; border-radius:4px;"&gt;',
                obj.static_featured
            )
        return "No image"
    image_preview.short_description = 'Preview'
```

9.5 Admin Site Customization

```
# MichaelAluminum/urls.py or admin.py
from django.contrib import admin

admin.site.site_header = "Michael Aluminum Admin"
admin.site.site_title = "Michael Aluminum"
admin.site.index_title = "Content Management"
```

Project Exercise 9: Open `projects/admin.py`. Add a custom action to bulk set `is_featured=True` on selected projects. Add an image preview column to the list view.

MODULE 10 — REST API with Django REST Framework

10.1 What is REST API?

A REST API lets other apps (mobile apps, React frontends, external services) access your data as JSON.

```
GET    /api/projects/          → List all projects
GET    /api/projects/1/        → Get project with id=1
POST   /api/projects/          → Create new project
PUT    /api/projects/1/        → Update project id=1
DELETE /api/projects/1/        → Delete project id=1
```

10.2 Install and Configure DRF

```
# settings.py
INSTALLED_APPS = [
    ...
    'rest_framework',
]

REST_FRAMEWORK = {
    'DEFAULT_PAGINATION_CLASS': 'rest_framework.pagination.PageNumberPagination',
    'PAGE_SIZE': 12,
    'DEFAULT_FILTER_BACKENDS': [
        'django_filters.rest_framework.DjangoFilterBackend',
        'rest_framework.filters.SearchFilter',
        'rest_framework.filters.OrderingFilter',
    ],
}
```

10.3 Serializers — Convert Models to JSON

```
# api/serializers.py
from rest_framework import serializers
from projects.models import Project, ProjectCategory
from services.models import Service


class ProjectCategorySerializer(serializers.ModelSerializer):
    class Meta:
        model = ProjectCategory
        fields = ['id', 'name', 'slug', 'icon']


class ProjectSerializer(serializers.ModelSerializer):
    category = ProjectCategorySerializer(read_only=True)
    category_id = serializers.IntegerField(write_only=True)

    class Meta:
        model = Project
        fields = [
            'id', 'title', 'slug', 'description',
            'category', 'category_id',
            'static_featured', 'static_before', 'static_after',
            'location', 'client', 'year',
            'is_active', 'is_featured',
        ]
        read_only_fields = ['slug']


class ServiceSerializer(serializers.ModelSerializer):
    class Meta:
        model = Service
        fields = ['id', 'title', 'slug', 'short_description', 'icon', 'static_image']
```

10.4 API Views

```
# api/views.py
from rest_framework import viewsets, filters
from rest_framework.decorators import api_view
from rest_framework.response import Response
from django_filters.rest_framework import DjangoFilterBackend
from projects.models import Project
from services.models import Service
from .serializers import ProjectSerializer, ServiceSerializer


# Simple function-based API view
@api_view(['GET'])
def api_overview(request):
    return Response({
        'projects': '/api/projects/',
        'services': '/api/services/',
        'categories': '/api/categories/',
    })


# ViewSet — handles all CRUD operations automatically
class ProjectViewSet(viewsets.ReadOnlyModelViewSet):
    """
    ReadOnlyModelViewSet = GET only (list + detail)
    ModelViewSet = full CRUD (GET, POST, PUT, DELETE)
    """
    queryset = Project.objects.select_related('category').filter(is_active=True)
    serializer_class = ProjectSerializer
    filter_backends = [DjangoFilterBackend, filters.SearchFilter, filters.OrderingFilter]
    filterset_fields = ['category__slug', 'year', 'is_featured']
    search_fields = ['title', 'description', 'location']
    ordering_fields = ['year', 'title']
    ordering = ['-year']

    def get_queryset(self):
        queryset = super().get_queryset()
        # Custom filter: ?category=handrail
        category = self.request.query_params.get('category')

        if category:
            queryset = queryset.filter(category__slug=category)
        return queryset


class ServiceViewSet(viewsets.ReadOnlyModelViewSet):
    queryset = Service.objects.select_related('category').filter(is_active=True)
    serializer_class = ServiceSerializer
```

10.5 API URLs with Router

```
# api/urls.py
from django.urls import path, include
from rest_framework.routers import DefaultRouter
from . import views

router = DefaultRouter()
router.register(r'projects', views.ProjectViewSet)
router.register(r'services', views.ServiceViewSet)

app_name = 'api'
urlpatterns = [
    path('', views.api_overview, name='overview'),
    path('', include(router.urls)),
]
```

10.6 Testing the API

```
# In your browser or tools like Postman/Insomnia:

# List all active projects
GET https://michael-aluminum.onrender.com/api/projects/

# Filter by category
GET https://michael-aluminum.onrender.com/api/projects/?category=handrail

# Search
GET https://michael-aluminum.onrender.com/api/projects/?search=bole

# Get single project
GET https://michael-aluminum.onrender.com/api/projects/1/

# List services
GET https://michael-aluminum.onrender.com/api/services/
```

Project Exercise 10: Open `api/views.py` and `api/urls.py`. Visit `/api/` in your browser. Add a new endpoint that returns the list of project categories.

MODULE 11 — Deployment

11.1 Development vs Production Settings

Never run `DEBUG=True` in production. Here is the difference:

Setting	Development	Production
`DEBUG`	`True`	`False`
`SECRET_KEY`	Any string	Long random string, from env var

Setting	Development	Production
`ALLOWED_HOSTS`	`['*']`	`['.onrender.com', 'yourdomain.com']`
`DATABASE`	SQLite	PostgreSQL
`EMAIL_BACKEND`	`console`	`smtp.EmailBackend`
`STATIC_FILES`	Django serves	WhiteNoise / CDN
`HTTPS`	Not required	Required

11.2 Environment Variables with python-decouple

Never hardcode secrets in settings.py.

```
# Install
pip install python-decouple
```

```
# MichaelAluminum/settings.py
from decouple import config, Csv

SECRET_KEY = config('SECRET_KEY')
DEBUG = config('DEBUG', default=False, cast=bool)
ALLOWED_HOSTS = config('ALLOWED_HOSTS', default='localhost', cast=Csv())
DATABASE_URL = config('DATABASE_URL', default=None)
```

```
# .env file (never commit this to git!)
SECRET_KEY=your-very-long-random-secret-key-here
DEBUG=True
ALLOWED_HOSTS=localhost,127.0.0.1
EMAIL_HOST_USER=michaeltadessemiki@gmail.com
EMAIL_HOST_PASSWORD=your-gmail-app-password
```

```
# .gitignore — must include:
.env
db.sqlite3
media/
staticfiles/
```

11.3 PostgreSQL with dj-database-url

```
pip install dj-database-url psycopg2-binary
```

```
# settings.py
import dj_database_url

DATABASE_URL = config('DATABASE_URL', default=None)

if DATABASE_URL:
    DATABASES = {
        'default': dj_database_url.parse(DATABASE_URL, conn_max_age=600)
    }
else:
    # SQLite for local development
    DATABASES = {
        'default': {
            'ENGINE': 'django.db.backends.sqlite3',
            'NAME': BASE_DIR / 'db.sqlite3',
        }
    }
```

11.4 Static Files with WhiteNoise

```
pip install whitenoise
```

```
# settings.py
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'whitenoise.middleware.WhiteNoiseMiddleware',  # Second position!
    ...
]

STATICFILES_STORAGE = 'whitenoise.storage.CompressedManifestStaticFilesStorage'
```

WhiteNoise:

- Serves static files directly from Django (no Nginx needed)
- Compresses files automatically (gzip)
- Sets long cache headers for performance

11.5 Production Security Settings

```
# settings.py — only active when DEBUG=False
if not DEBUG:
    # Force HTTPS
    SECURE_SSL_REDIRECT = True

    # HTTP Strict Transport Security
    SECURE_HSTS_SECONDS = 31536000       # 1 year
    SECURE_HSTS_INCLUDE_SUBDOMAINS = True
    SECURE_HSTS_PRELOAD = True

    # Prevent XSS attacks
    SECURE_BROWSER_XSS_FILTER = True
    SECURE_CONTENT_TYPE_NOSNIFF = True

    # Prevent Clickjacking
    X_FRAME_OPTIONS = 'DENY'

    # Secure cookies (HTTPS only)
    SESSION_COOKIE_SECURE = True
    CSRF_COOKIE_SECURE = True
```

11.6 Gunicorn — WSGI Server

Django's development server is NOT suitable for production.

Gunicorn is a production-grade WSGI server.

```
pip install gunicorn
```

```
# Run with gunicorn
gunicorn MichaelAluminum.wsgi:application \
    --bind 0.0.0.0:$PORT \
    --workers 2 \
    --timeout 120 \
    --log-level info
```

- `MichaelAluminum.wsgi:application` — points to `wsgi.py` in the project folder
- `--bind 0.0.0.0:$PORT` — listen on all interfaces, use $PORT from environment
- `--workers 2` — 2 worker processes (free plan: keep at 2)
- `--timeout 120` — kill workers that take longer than 120 seconds

11.7 Dockerfile — Containerization

The Michael Aluminum project uses Docker on Render:

```
# Dockerfile
FROM python:3.13-slim           # Base image — Python 3.13

# Environment variables
ENV PYTHONUNBUFFERED=1          # No output buffering
ENV PYTHONDONTWRITEBYTECODE=1   # No .pyc files
ENV SECRET_KEY=build-time-dummy # Dummy key for collectstatic
ENV DEBUG=False
ENV ALLOWED_HOSTS=*

WORKDIR /app                    # Working directory

# Install system dependencies
RUN apt-get update &amp;&amp; \
    apt-get install -y postgresql-client &amp;&amp; \
    rm -rf /var/lib/apt/lists/*

# Install Python dependencies
COPY requirements.txt .
RUN pip install --upgrade pip &amp;&amp; \
    pip install --no-cache-dir -r requirements.txt

# Copy project files
COPY . .

# Collect static files at build time
RUN python manage.py collectstatic --noinput

# Startup script
COPY docker_start.sh /docker_start.sh
RUN chmod +x /docker_start.sh

EXPOSE 10000

CMD ["/bin/sh", "/docker_start.sh"]
```

11.8 docker_start.sh — Startup Script

```
#!/bin/sh
set -e    # Exit immediately if any command fails

echo "Running migrations..."
python manage.py migrate --noinput

echo "Seeding data..."
python manage.py seed_all_data
python manage.py seed_services
python manage.py seed_projects

echo "Creating superuser..."
python manage.py shell -c "
from django.contrib.auth import get_user_model
User = get_user_model()
if not User.objects.filter(username='admin').exists():
    User.objects.create_superuser('admin', 'email@example.com', 'password')
"

echo "Starting gunicorn..."
exec gunicorn MichaelAluminum.wsgi:application \
    --bind 0.0.0.0:${PORT:-10000} \
    --workers 2 \
    --timeout 120
```

11.9 requirements.txt

Every package your project needs:

```
Django==5.0
djangorestframework==3.14.0
django-filter==24.1
Pillow==11.0.0              # For ImageField
python-decouple==3.8        # For environment variables
django-cors-headers==4.3.1
django-crispy-forms==2.3
crispy-bootstrap5==2024.10
gunicorn==21.2.0            # Production web server
psycopg2-binary==2.9.12     # PostgreSQL adapter
whitenoise==6.6.0           # Static file serving
dj-database-url==2.1.0      # Parse DATABASE_URL string
```

11.10 Deploying to Render — Step by Step

```
1. Push code to GitHub
   git add .
   git commit -m "Ready for deployment"
   git push origin main

2. Create PostgreSQL on Render
   Dashboard → New → PostgreSQL → Free plan
   Copy the Internal Database URL

3. Create Web Service on Render
   Dashboard → New → Web Service
   Connect GitHub repo
   Runtime: Docker (auto-detected from Dockerfile)

4. Set Environment Variables
   SECRET_KEY      → generate random
   DEBUG           → false
   ALLOWED_HOSTS   → .onrender.com
   DATABASE_URL    → paste Internal Database URL
   CSRF_TRUSTED_ORIGINS → https://your-app.onrender.com

5. Deploy
   Click Create Web Service
   Wait 3-5 minutes
   Watch logs for errors
```

11.11 Understanding the Render Free Plan

Feature	Free Plan	Paid ($7/month)
Always on	■ (spins down after 15 min)	■
Cold start	~30 seconds	None
RAM	512MB	512MB+
Build minutes	500/month	Unlimited
Persistent disk	■	Available

How to keep free plan always on: Use UptimeRobot to ping `/health/` every 5 minutes.

Project Exercise 11: Read through the Michael Aluminum `Dockerfile` and `docker_start.sh` line by line. Understand what each line does. Then look at `MichaelAluminum/settings.py` and identify every environment variable.

MODULE 12 — Advanced Topics

12.1 Management Commands

Custom commands you can run with `python manage.py`.

The Michael Aluminum project uses several:

```
# services/management/commands/seed_services.py
from django.core.management.base import BaseCommand
from services.models import ServiceCategory, Service

class Command(BaseCommand):
    help = 'Seed services and categories into database'

    def add_arguments(self, parser):
        # Optional argument: --reset
        parser.add_argument('--reset', action='store_true',
                            help='Delete existing services first')

    def handle(self, *args, **options):
        if options['reset']:
            Service.objects.all().delete()
            self.stdout.write('Services deleted.')

        # Create category
        category, created = ServiceCategory.objects.get_or_create(
            name='Aluminum Works',
            defaults={'order': 1}
        )

        # Create service
        service, created = Service.objects.update_or_create(
            slug='handrail',
            defaults={
                'title': 'Handrail',
                'category': category,
                'is_active': True,
            }
        )

        # Use self.stdout.write for output
        self.stdout.write(
            self.style.SUCCESS(f'Done! {Service.objects.count()} services.')
        )
        # self.style.SUCCESS = green

        # self.style.WARNING = yellow
        # self.style.ERROR   = red
```

```
# Run it
python manage.py seed_services
python manage.py seed_services --reset
```

12.2 Signals

Signals let you run code automatically when something happens.

```
# projects/signals.py
from django.db.models.signals import post_save, pre_delete
from django.dispatch import receiver
from .models import Project

# Runs AFTER a Project is saved
@receiver(post_save, sender=Project)
def project_saved(sender, instance, created, **kwargs):
    if created:
        print(f"New project created: {instance.title}")
    else:
        print(f"Project updated: {instance.title}")

# Runs BEFORE a Project is deleted
@receiver(pre_delete, sender=Project)
def project_deleted(sender, instance, **kwargs):
    print(f"Project being deleted: {instance.title}")
```

```
# projects/apps.py — must connect signals
from django.apps import AppConfig

class ProjectsConfig(AppConfig):
    name = 'projects'

    def ready(self):
        import projects.signals  # Connect signals when app starts
```

12.3 Email Sending

```
# settings.py
EMAIL_BACKEND = 'django.core.mail.backends.smtp.EmailBackend'
EMAIL_HOST = 'smtp.gmail.com'
EMAIL_PORT = 587
EMAIL_USE_TLS = True
EMAIL_HOST_USER = config('EMAIL_HOST_USER')
EMAIL_HOST_PASSWORD = config('EMAIL_HOST_PASSWORD')
DEFAULT_FROM_EMAIL = 'Michael Aluminum &lt;noreply@michaelalumin.com&gt;'
```

```
# views.py — send email when contact form submitted
from django.core.mail import send_mail, EmailMultiAlternatives
from django.template.loader import render_to_string

def contact(request):
    if request.method == 'POST':
        name = request.POST.get('name')
        email = request.POST.get('email')
        message = request.POST.get('message')

        # Simple email
        send_mail(
            subject=f'New Contact: {name}',
            message=f'From: {email}\n\n{message}',
            from_email=settings.DEFAULT_FROM_EMAIL,
            recipient_list=['michaeltadessemiki@gmail.com'],
            fail_silently=False,
        )

        # HTML email
        html_content = render_to_string('emails/contact.html', {
            'name': name,
            'email': email,
            'message': message,
        })
        msg = EmailMultiAlternatives(
            subject=f'New Contact: {name}',
            body=message,
            from_email=settings.DEFAULT_FROM_EMAIL,
            to=['michaeltadessemiki@gmail.com'],
        )
        msg.attach_alternative(html_content, "text/html")
        msg.send()
```

Gmail App Password: *Go to https://myaccount.google.com/apppasswords to generate a password specifically for this app.*

12.4 Sitemap & SEO

```
# settings.py
INSTALLED_APPS += ['django.contrib.sitemaps']
```

```
# core/sitemaps.py
from django.contrib.sitemaps import Sitemap
from projects.models import Project

class ProjectSitemap(Sitemap):
    changefreq = 'weekly'
    priority = 0.8
    protocol = 'https'

    def items(self):
        return Project.objects.filter(is_active=True)

    def location(self, obj):
        return f'/projects/{obj.slug}/'

    def lastmod(self, obj):
        return obj.updated_at
```

```
# urls.py
from django.contrib.sitemaps.views import sitemap

sitemaps = {'projects': ProjectSitemap}

urlpatterns += [
    path('sitemap.xml', sitemap, {'sitemaps': sitemaps}),
    path('robots.txt', robots_txt_view),
]
```

12.5 Caching

```
# Cache entire view for 15 minutes
from django.views.decorators.cache import cache_page

@cache_page(60 * 15)
def projects_list(request):
    ...

# Cache per-user
from django.views.decorators.vary import vary_on_cookie

@cache_page(60 * 15)
@vary_on_cookie
def my_profile(request):
    ...

# Cache template fragment
# In template:
{% load cache %}
{% cache 900 project_list %}
    &lt;!-- Cached for 900 seconds --&gt;
    {% for project in projects %}...{% endfor %}
{% endcache %}

# Low-level cache API
from django.core.cache import cache

# Store
cache.set('my_key', my_data, timeout=300)

# Retrieve
data = cache.get('my_key')

# Delete
cache.delete('my_key')
```

12.6 Performance Optimization

```
# 1. select_related — for ForeignKey (one JOIN query)
Project.objects.select_related('category').all()

# 2. prefetch_related — for ManyToMany (separate query)
Project.objects.prefetch_related('gallery_images').all()

# 3. only() — load only specific fields
Project.objects.only('title', 'slug', 'static_featured')

# 4. defer() — load everything EXCEPT these fields
Project.objects.defer('description', 'challenges', 'solutions')

# 5. values() — returns dicts instead of objects (faster)
Project.objects.values('title', 'slug', 'year')

# 6. values_list() — returns tuples
Project.objects.values_list('title', 'slug')

# 7. count() — COUNT(*) SQL, faster than len()
Project.objects.filter(is_active=True).count()

# 8. exists() — faster than count() &gt; 0
Project.objects.filter(slug='handrail').exists()

# 9. Bulk operations
Project.objects.bulk_create([
    Project(title='P1', slug='p1', year=2025),
    Project(title='P2', slug='p2', year=2025),
])

Project.objects.filter(year=2024).update(is_active=False)
```

12.7 Writing Tests

```
# projects/tests.py
from django.test import TestCase, Client
from django.urls import reverse
from .models import Project, ProjectCategory


class ProjectModelTest(TestCase):

    def setUp(self):
        # Runs before each test
        self.category = ProjectCategory.objects.create(
            name='Handrail', slug='handrail', order=1
        )
        self.project = Project.objects.create(
            title='Test Handrail Project',
            slug='test-handrail-project',
            category=self.category,
            description='Test description',
            location='Addis Ababa',
            client='Test Client',
            year=2025,
            materials_used='Aluminum',
            static_featured='images/test.jpg',
        )

    def test_project_str(self):
        self.assertEqual(str(self.project), 'Test Handrail Project')

    def test_project_is_active_default(self):
        self.assertTrue(self.project.is_active)

    def test_project_category(self):
        self.assertEqual(self.project.category.name, 'Handrail')


class ProjectViewTest(TestCase):

    def setUp(self):
```

```
        self.client = Client()
        self.category = ProjectCategory.objects.create(
            name='Handrail', slug='handrail', order=1
        )
        self.project = Project.objects.create(
            title='Test Project',
            slug='test-project',
            category=self.category,
            description='Test',
            location='Addis Ababa',
            client='Client',
            year=2025,
            materials_used='Aluminum',
            static_featured='images/test.jpg',
        )

    def test_projects_list_page(self):
        response = self.client.get(reverse('projects:list'))
        self.assertEqual(response.status_code, 200)
        self.assertContains(response, 'Test Project')

    def test_project_detail_page(self):
        response = self.client.get(
            reverse('projects:detail', kwargs={'slug': 'test-project'})
        )
        self.assertEqual(response.status_code, 200)
        self.assertContains(response, 'Test Project')

    def test_project_detail_404(self):
        response = self.client.get('/projects/does-not-exist/')
        self.assertEqual(response.status_code, 404)
```

```
# Run all tests
python manage.py test

# Run tests for specific app
python manage.py test projects

# Run with verbosity
python manage.py test --verbosity=2
```

12.8 Internationalization (i18n)

The Michael Aluminum project supports English, Amharic, and Tigrinya.

```
# settings.py
USE_I18N = True
LANGUAGE_CODE = 'en-us'
LANGUAGES = [
    ('en', 'English'),
    ('am', 'Amharic'),
    ('ti', 'Tigrinya'),
]
LOCALE_PATHS = [BASE_DIR / 'locale']
```

```
# In views.py or models.py
from django.utils.translation import gettext_lazy as _

class Service(models.Model):
    title = models.CharField(_('Title'), max_length=200)

    class Meta:
        verbose_name = _('Service')
        verbose_name_plural = _('Services')
```

```
&lt;!-- In templates --&gt;
{% load i18n %}
&lt;h1&gt;{% trans "Our Services" %}&lt;/h1&gt;
&lt;p&gt;{% trans "World-class solutions" %}&lt;/p&gt;

&lt;!-- Pluralization --&gt;
{% blocktrans count count=project_count %}
    {{ count }} project
{% plural %}
    {{ count }} projects
{% endblocktrans %}
```

FINAL PROJECT CHALLENGES

Test your knowledge by completing these tasks on the Michael Aluminum project:

Challenge 1 — Beginner

Add a `views_count` field to the `Project` model that increments by 1 every time someone views a project detail page.

Challenge 2 — Intermediate

Create a `Newsletter` model and form. When someone subscribes, send them a welcome email and save their email to the database.

Challenge 3 — Intermediate

Add pagination to the projects list page. Show 6 projects per page with Previous/Next links.

Challenge 4 — Advanced

Create a REST API endpoint `/api/stats/` that returns:

```
{
  "total_projects": 6,
  "total_services": 6,
  "total_testimonials": 5,
  "categories": ["Handrail", "LTZ Windows", "..."]
}
```

Challenge 5 — Advanced

Add a search feature to the home page that searches across projects, services, and blog posts simultaneously and displays results on a dedicated search results page at `/search/?q=handrail`.

QUICK REFERENCE — Django Cheat Sheet

Shell Commands

```
python manage.py runserver          # Start dev server
python manage.py makemigrations     # Create migrations
python manage.py migrate            # Apply migrations
python manage.py createsuperuser    # Create admin user
python manage.py shell              # Open Django shell
python manage.py collectstatic      # Gather static files
python manage.py check              # Check for issues
python manage.py test               # Run tests
```

QuerySet Cheat Sheet

```
Model.objects.all()                 # All objects
Model.objects.filter(field=value)   # Filter
Model.objects.exclude(field=value)  # Exclude
Model.objects.get(pk=1)             # Single object
Model.objects.create(field=value)   # Create
Model.objects.first()               # First object
Model.objects.last()                # Last object
Model.objects.count()               # Count
Model.objects.exists()              # Bool check
Model.objects.order_by('field')     # Sort ascending
Model.objects.order_by('-field')    # Sort descending
Model.objects.select_related('fk')  # JOIN query
Model.objects.values('f1', 'f2')    # Return dicts
Model.objects.distinct()            # Remove duplicates
Model.objects.update(field=value)   # Bulk update
Model.objects.delete()              # Bulk delete
```

Template Tags Cheat Sheet

```
{{ variable }}              Output variable
{{ var|filter }}            Apply filter
{% for x in list %}         Loop
{% if condition %}          Conditional
{% url 'name' slug %}       URL
{% static 'path' %}         Static file
{% load static %}           Load tag library
{% extends 'base.html' %}   Inherit template
{% block name %}{% endblock %} Define block
{% include 'partial.html' %} Include template
{% csrf_token %}            CSRF protection
```

HTTP Status Codes

```
200 OK              — Success
201 Created         — Resource created
301 Moved           — Permanent redirect
302 Found           — Temporary redirect
400 Bad Request     — Invalid input
401 Unauthorized    — Login required
403 Forbidden       — No permission
404 Not Found       — Resource not found
500 Server Error    — Django/Python error
```

LEARNING PATH

```
Week 1:  Module 1  — Python Fundamentals
Week 2:  Module 2  — Django Basics
Week 3:  Module 3  — Models &amp;amp; Database
Week 4:  Module 4  — Views &amp;amp; URLs
Week 5:  Module 5  — Templates
Week 6:  Module 6  — Forms &amp;amp; User Input
Week 7:  Module 7  — Static Files &amp;amp; Media
Week 8:  Module 8  — Authentication
Week 9:  Module 9  — Admin Customization
Week 10: Module 10 — REST API
Week 11: Module 11 — Deployment
Week 12: Module 12 — Advanced Topics
         Final Challenges
```

RESOURCES

- **Django Official Docs:** https://docs.djangoproject.com/
- **Django REST Framework:** https://www.django-rest-framework.org/
- **Python Docs:** https://docs.python.org/3/

- **Bootstrap 5:** https://getbootstrap.com/
- **Render Deployment:** https://render.com/docs/deploy-django
- **GitHub:** https://github.com/getachewzemichael/michael-aluminum

---

This guide was created using the Michael Aluminum and Glass Technology project as a real-world learning example.

Last updated: July 2026